

DSA STUDY BLUEPRINT SERIES

118. Bellman Ford Algorithm Single Source Shortest Path DP

Shortest Path with Negative Weights (Bellman-Ford)

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Bellman-Ford:** Relax all edges $V-1$ times.
- Detect negative cycles by running a final relaxation step.

Memory & Execution Blueprint

Final relaxation step updates distance $\Rightarrow$ negative cycle exists!

C++ Implementation Reference

Standard code layout and syntax

```
vector bellmanFord(int V, vector<edge>& edges, int src) {  
    vector<int> dist(V, 1e9); dist[src] = 0;  
    for (int i = 1; i <= V - 1; i++) {  
        for (auto edge : edges) {  
            int u = edge[0], v = edge[1], w = edge[2];  
            if (dist[u] != 1e9 && dist[u] + w < dist[v]) dist[v] = dist[u] + w;  
        }  
    }  
    return dist;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(V * E)$
Space Complexity	$O(V)$

Key Practices & Gotchas

- **Important:** Bellman-Ford relaxes edges $V-1$ times because a simple path can have at most $V-1$ edges.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.